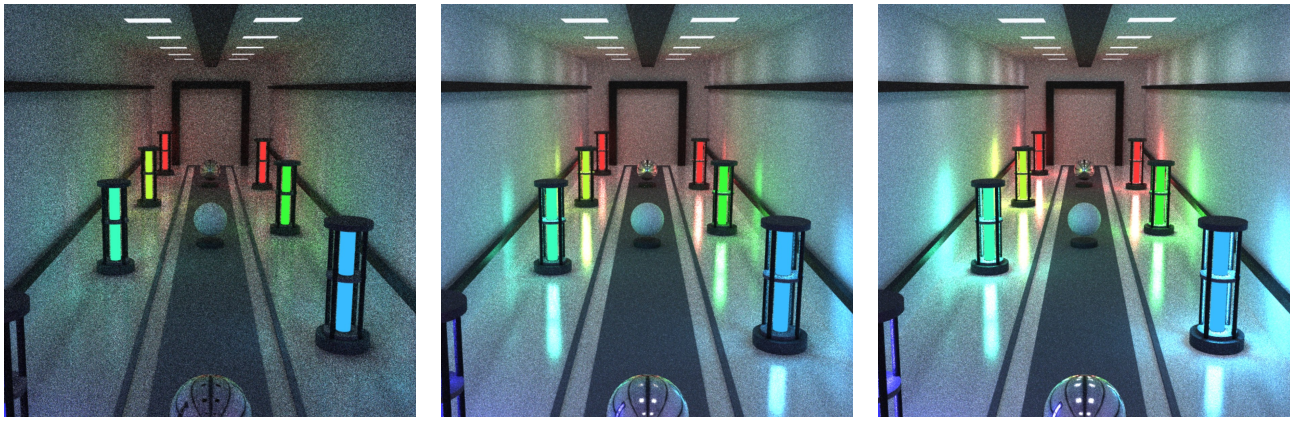
Reservoir-Based Spatiotemporal Importance Resampling Implementation

Arthur Carvalho Guerra

Institut Polytechnique de Paris - IGD Master

March 2026



(a) Uniform light sampling (b) ReSTIR (biased) (c) ReSTIR (unbiased)

Figure 1: Accumulated comparison over 200 frames between the baseline uniform light sampling and the biased and unbiased ReSTIR implementations in a scene with 672 emissive triangles. All images use 1 sample per pixel and 30 bounces per ray. Streaming RIS is used for subsequent bounces in the ReSTIR configurations.

1 Introduction

Physically based rendering aims to simulate the transport of light in a scene in order to produce realistic images. Among the different approaches, path tracing has become a standard method due to its generality and its ability to accurately reproduce complex lighting effects such as soft shadows, reflections, and global illumination. However, path tracing relies on Monte Carlo integration, which inherently introduces noise under practical computational constraints.

In real applications, the number of samples per pixel, the maximum number of light bounces, and the overall rendering time must all be limited. These constraints prevent full convergence of the estimator, resulting in noisy images, especially in challenging lighting conditions. Reducing this noise while staying within a fixed computational budget remains one of the main challenges in image synthesis.

A particularly challenging component of path tracing is the estimation of direct lighting in scenes containing many light sources. As the number of lights increases, sampling lights uniformly or even with simple importance sampling strategies becomes inefficient, as only a small subset of light samples significantly contributes to the final image. This leads to high variance and visible noise. The problem is even more critical in real-time rendering, where only a very limited number of rays can be cast per frame.

The method proposed in *Spatiotemporal Reservoir Resampling for Real-Time Ray Tracing with Dynamic Direct Lighting* (ReSTIR) [1] introduces a novel approach to address this problem by combining resampled importance sampling with reservoir sampling and sample reuse across both space and time. Instead of treating each pixel independently, ReSTIR reuses light samples from neighboring pixels and previous frames, effectively increasing the number of candidate samples without increasing the number of ray evaluations. This leads to significantly improved image quality at real-time performance.

This project presents an implementation of the ReSTIR algorithm within a WebGPU path tracer. The implementation follows the original method closely, including candidate generation, reservoir-based resampling, and both temporal and spatial reuse of samples.

The goal of this report is to describe the principles

behind ReSTIR, detail its implementation in a real-time rendering context, and evaluate its impact on both visual quality and performance.

2 Background and Theory

2.1 Direct Lighting as Monte Carlo Integration

In path tracing, the outgoing radiance at a surface point is expressed as an integral over all incoming light directions. When focusing on direct lighting, this integral reduces to contributions coming directly from light sources. In practice, this integral is estimated using Monte Carlo sampling by selecting a small number of light samples and evaluating their contribution.

Each sample consists of selecting a point on a light source, evaluating the incoming radiance, and checking its visibility. The final estimate is obtained by averaging these contributions, weighted by the inverse of the sampling probability. While this approach is unbiased, it suffers from high variance when only a few samples are used, especially in scenes with many light sources or complex visibility.

In this project, the baseline used for comparison is a simple uniform light sampling strategy, where a single light is selected at random for each shading point. This approach is straightforward to implement but becomes inefficient as the number of lights increases, since most sampled lights contribute little or nothing to the final result.

Importance sampling aims to reduce this variance by favoring samples that are expected to contribute more to the final result. However, designing an effective importance sampling strategy in this context is challenging. The contribution of each light depends on several factors, including scene geometry, visibility, and material properties at the shading point. As a result, it is difficult to define a sampling distribution that reliably captures the most important light contributions, especially in scenes with many lights.

2.2 Resampled Importance Sampling (RIS)

Resampled Importance Sampling (RIS) is a technique that improves sampling efficiency when the target distribution is difficult to sample from directly. Instead of drawing samples from this complex distribution, RIS samples from a simpler proposal distribution and compensates for the difference using importance weighting. This allows approximating the desired distribution without explicitly constructing it.

RIS works by drawing a set of candidate samples from the proposal distribution, each associated with a weight proportional to the ratio between the target and proposal distributions. For a candidate sample x_i, the weight is defined as:

$$w_i = \frac{\hat{p}(x_i)}{p(x_i)}$$

where $\hat{p}(x)$ is the target distribution and $p(x)$ is the proposal distribution.

One candidate is then selected with probability proportional to its weight, and its contribution is adjusted accordingly. Under appropriate conditions, the resulting estimator converges to the correct solution as the number of candidates increases.

This approach makes it possible to approximate complex sampling strategies using only simple sampling procedures. However, a naive implementation would require storing and processing all candidate samples, which becomes impractical in real-time applications.

2.3 Reservoir Sampling

Reservoir sampling provides an efficient way to implement RIS while using constant memory. Instead of storing all candidate samples, it maintains a compact structure called a reservoir, which represents a weighted selection among all candidates seen so far. A reservoir stores at least:

- a selected sample y,
- the sum of weights of all candidates seen so far w_{sum},
- the number of candidates seen.

As new candidates are processed, the reservoir is updated incrementally: each candidate x_i has a probability of replacing the current selected sample proportional to w_i/w_{sum}, where w_i is the weight associated to x_i. This allows the reservoir to effectively sample from the underlying weighted distribution without needing to know all candidates in advance.

At the end of the process, the reservoir contains a single representative sample whose selection is consistent with the distribution defined by the accumulated weights.

This approach makes it possible to handle a large number of candidates efficiently, which is essential for real-time applications.

3 Method Overview and Implementation

The ReSTIR algorithm is implemented as a sequence of compute passes that progressively build and refine a set of light samples for each pixel. The process begins by tracing primary camera rays to identify visible surfaces, followed by the generation of initial candidate samples using Resampled Importance Sampling (RIS). These samples are then refined through a series of reuse steps, where information is shared across time and neighboring pixels using reservoir sampling. Finally, the selected samples are used during shading to estimate direct lighting.

In this implementation, a single reservoir is maintained per pixel ($N = 1$), and only one sample is used

to estimate direct lighting at the primary hit. This follows the standard formulation of ReSTIR, where the goal is mainly to maximize the quality of the samples within a fixed sample budget. Using SPP > 1 while sharing a single reservoir across samples would introduce bias, as shown below.

Using the notation of the paper, let y_q be the selected sample for pixel q, with corresponding reservoir weight W_q. The estimator of the direct lighting integral I_q is given by

$$\hat{I}_q = f(y_q)W_q,$$

where $f(y_q)$ denotes the contribution of the chosen light sample at the shading point. If the estimator is unbiased, then

$$\mathbb{E}[\hat{I}_q] = I_q.$$

If we want multiple samples per pixel, unbiasedness can be preserved by performing independent executions of the algorithm and averaging the resulting estimators. Denoting by $\hat{I}_q^{(s)}$ the estimator obtained from the s-th execution, we obtain

$$\bar{I}_q = \frac{1}{S}\sum_{s=1}^{S}\hat{I}_q^{(s)}, \qquad \mathbb{E}[\bar{I}_q] = \frac{1}{S}\sum_{s=1}^{S}\mathbb{E}[\hat{I}_q^{(s)}] = I_q.$$

Meanwhile, if multiple camera samples within the same pixel were to share a single reservoir, then different shading points would reuse the same selected sample y_q and weight W_q, even though their target distributions $f_s(y_q)$ are potentially different. In that case, the estimator would no longer correspond to the correct per-sample expectation, and bias would be introduced.

For this reason, the implementation is restricted to the $N = 1$ case without averaging multiple ReSTIR passes.

The different stages of the algorithm, along with their implementation details, are described in the following sections.

3.1 Primary Visibility Pass

The first step consists in tracing a single primary ray per pixel to determine the visible surface. For each pixel, the closest non-emissive surface hit is stored in a buffer, along with the necessary geometric information such as position and normal.

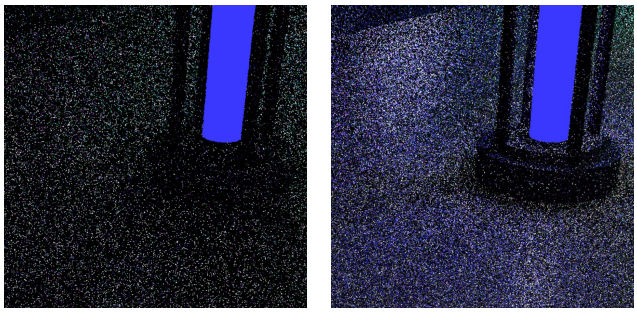
This information is reused in all subsequent passes, as it defines the shading point at which direct lighting is evaluated. By separating this step from the rest of the pipeline, the algorithm avoids recomputing primary hits during the reuse stages.

3.2 Initial RIS Pass

In this pass, an initial reservoir is constructed independently for each pixel using RIS. A set of candidate light samples is generated, and one of them is selected based on their relative weights.

In this implementation, light sources are represented as emissive triangles. Candidate samples are generated by first selecting an emissive triangle and then sampling a point on its surface using barycentric coordinates. Each sample therefore corresponds to a point on an emissive surface, along with the information required to evaluate its contribution in later stages.

The result of this pass is a local reservoir per pixel, containing a single selected light sample. At this stage, no reuse is performed, and each pixel is processed independently.



(a) Uniform sampling (b) RIS only

Figure 2: Uniform sampling vs. Streaming RIS only.

3.3 Visibility Reuse Pass

The initial reservoirs are filtered based on visibility. For each pixel, a shadow ray is traced toward the selected light sample stored in the reservoir. If the light is occluded, the reservoir is marked as inactive and discarded from further reuse. This step is important to prevent invalid samples from propagating during spatial and temporal reuse.

3.4 Temporal Reuse Pass

Temporal reuse combines the current reservoir of each pixel with a reservoir from a previous frame. This allows reusing samples generated in earlier frames, effectively increasing the number of candidates available at each pixel.

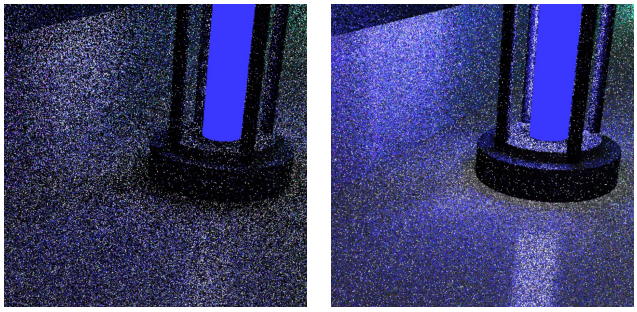
In this implementation, the temporal neighbor is obtained by reprojecting the current pixel into the previous frame using the current and previous camera view matrices. Since the geometry and lighting remain static in the scene, no additional per-object motion information is needed. This allows retrieving the corresponding pixel from the previous frame without requiring explicit motion vectors, as mentioned in the paper. The reprojected pixel is then used to index a buffer storing the primary hit surface of the previous frame.

Both biased and unbiased formulations of temporal reuse are implemented. The biased version directly combines the two reservoirs and computes the final weight W using the sum of their sample count M.

Meanwhile, the unbiased version accounts for the visibility of the selected sample in both pixels, found by casting additional shadow rays. In particular, the normalization term is computed by accumulating the number of candidates M only for reservoirs where the sample is valid (visible and with $\hat{p}_{q_i}(y) \neq 0$), forming the set Z. If the final selected sample is not visible from the current pixel, the resulting reservoir is marked as invalid.

To prevent overly strong contributions from long temporal histories, the number of candidates of the temporal reservoir is clamped to at most 20 times that of the current reservoir, as described in the original paper.

The combination of reservoirs is performed using reservoir sampling, ensuring that the resulting sample remains consistent with the weighted distribution of all candidates seen so far.



(a) RIS only (b) Biased temporal reuse

Figure 3: Streaming RIS only vs. with temporal reuse.

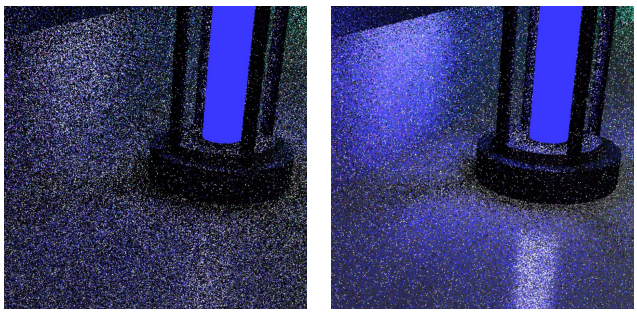
3.5 Spatial Reuse Pass

Spatial reuse extends the reuse process to neighboring pixels. For each pixel, a small set of neighboring pixels is selected within a fixed radius, and their reservoirs are combined with the current one.

In this implementation, spatial reuse follows the same strategy and parameters as described in the original paper. Neighboring pixels are selected within a fixed radius, and compatibility between pixels is enforced following the criteria described in the original paper. The biased version uses five neighbors and two reuse passes, while the unbiased version uses three neighbors and a single pass.

Again, both biased and unbiased formulations are implemented. The process follows the same logic as in temporal reuse, where reservoirs are combined and weighted according to their sample counts M. In the unbiased version, the validity of the selected sample must also be evaluated at each neighboring pixel. This requires checking visibility and verifying that $\hat{p}_{q_i}(y) \neq 0$ for each neighbor i, in order to correctly construct the normalization set Z.

This step significantly increases the effective number of candidates contributing to each pixel, leading to a substantial reduction in noise.



(a) RIS only (b) Biased spatial reuse

Figure 4: Streaming RIS only vs. with spatial reuse.

3.6 Shading Pass

The final reservoir obtained after the reuse passes is used during shading to estimate direct lighting at the primary surface.

For the first bounce, the selected light sample stored in the reservoir is used directly. For subsequent bounces, the algorithm behaves like a standard path tracer. Direct lighting at these later bounces is computed using either uniform light sampling or a streaming RIS estimator.

The use of streaming RIS at later bounces is not part of the original method described in the paper, but was added as an option in this implementation. While it can improve sampling quality, it also increases the computational cost, especially when using higher maximum ray depths.

The final image is then displayed, with optional frame accumulation and tone mapping.

4 Implementation Details

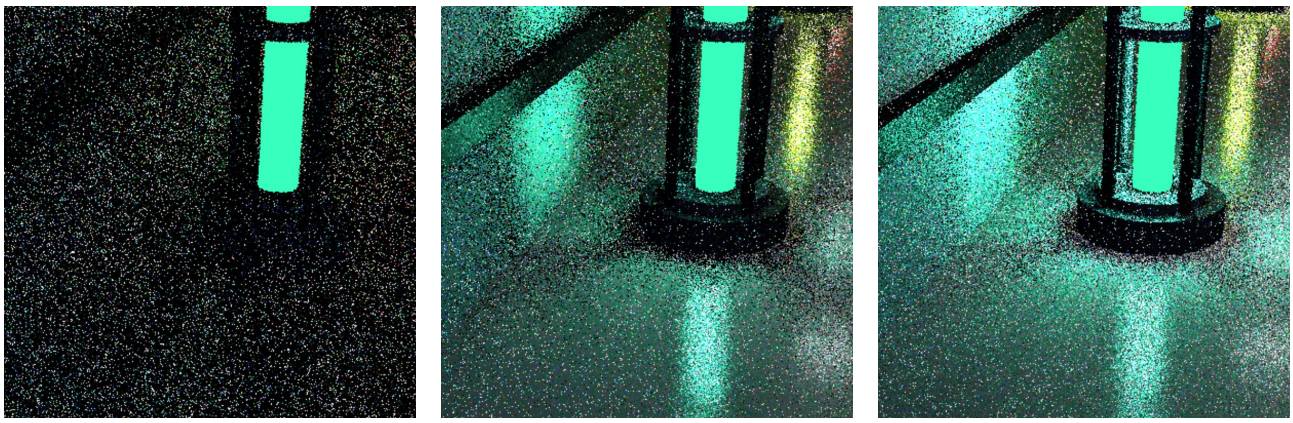
4.1 Data representation and Pipeline

The implementation is based on GPU buffers storing both scene data and intermediate results across the different passes. Geometry is represented using indexed triangle meshes and accelerated using a BVH for ray intersections. Light sources are represented as emissive triangles, which are sampled using barycentric coordinates to generate candidate points on their surface.

In addition to the reservoirs, the primary hit surface is stored for both the current and previous frames. This information includes the position, normal, and camera distance of the surface, and is used during temporal and spatial reuse to reproject pixels and evaluate sample compatibility across frames and neighboring pixels.

Each pixel stores a single reservoir ($N = 1$). In addition to the standard reservoir quantities, the implementation stores the final reservoir weight W, the value of the target function $\hat{p}$ at the shading point, and an activity flag indicating whether the reservoir is valid. These additional values simplify reuse operations and allow invalid reservoirs to be skipped efficiently.

Compatibility between pixels during temporal and spatial reuse is enforced using thresholds on geometric



(a) Uniform light sampling (b) ReSTIR (biased) (c) ReSTIR (unbiased)

Figure 5: Single-frame comparison between uniform light sampling and the biased and unbiased ReSTIR implementations. All images use 1 sample per pixel and 3 bounces per ray.

properties. In particular, neighboring samples are only considered if their surface normals differ by less than 25° and their camera distances differ by less than 10%.

Data from the current and previous frames, including both reservoirs and primary surface information, is alternated between buffers across passes and frames to allow safe updates during reuse operations. In practice, several bind groups (four in total) are used to manage this data flow, enabling efficient swapping between frames and between shader stages.

4.2 Parameters and practical choices

All parameters used in the implementation follow those described in the original paper. While some variations were tested, the original configuration consistently provided stable and good-quality results. In particular, the number of candidates used in the RIS stage is set to $M = 32$, which provided a good balance between quality and performance.

Practical choices were made to reduce computation during reuse passes. In particular, storing additional values such as the final reservoir weight and the target function value avoids repeated evaluations when combining reservoirs.

5 Results

The method was evaluated on a scene containing 672 emissive triangles (cylindrical and square lights). This setup creates a challenging many-light scenario where uniform sampling becomes inefficient.

Figure 1 shows a longer accumulated render comparing uniform light sampling with the biased and unbiased variants. These results show that ReSTIR converges significantly faster than the baseline, as uniform sampling fails to efficiently capture the contributions of most light sources across the scene.

Overall, ReSTIR provides a clear visual improvement. Uniform sampling produces a very noisy image, while both biased and unbiased ReSTIR significantly reduce noise and produce stable lighting even with a single sample per pixel. This difference is visible in Figure 5, where the noise in the uniform sampling result is clearly visible compared to the more precise illumination obtained with ReSTIR.

In terms of performance, the average frame time without ReSTIR is 57 ms on an RTX 4050 laptop GPU. Enabling biased ReSTIR increases this to 69.5 ms, corresponding to an overhead of about 13 ms, which remains consistent for higher maximum ray depths, since the additional cost occurs only at the first ray hit. The unbiased version reaches 129.3 ms, introducing an additional cost of roughly 60 ms, mainly due to the shadow rays required for visibility checks and normalization.

As expected, the biased version introduces a slight darkening, particularly near geometric discontinuities. The unbiased version removes these artifacts at the cost of increased computation time. This can be observed in Figure 5, for example at the base of the cylindrical lights and along the vertical rods.

6 Conclusion

This project presented a WebGPU implementation of ReSTIR for direct lighting in a many-light path tracing setting. The method follows the original paper closely and shows a clear improvement in image quality compared to uniform light sampling, with a relatively small overhead in the biased case. The results also confirm the main tradeoff described in the paper: the biased version is more efficient, while the unbiased version removes darkening artifacts at a significantly higher cost.

References

[1] Benedikt Bitterli, Chris Wyman, Matt Pharr, Peter Shirley, Aaron Lefohn, and Wojciech Jarosz. Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting. *ACM Transactions on Graphics (Proceedings of SIGGRAPH)*, 39(4), July 2020.